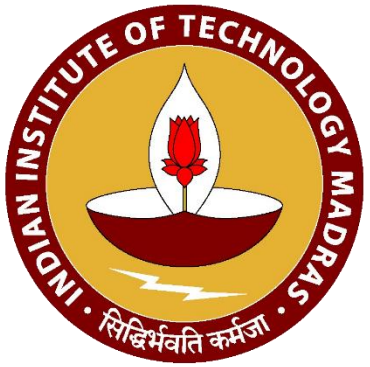
Decision Trees and their Variants

DA5400: Foundations of Machine Learning

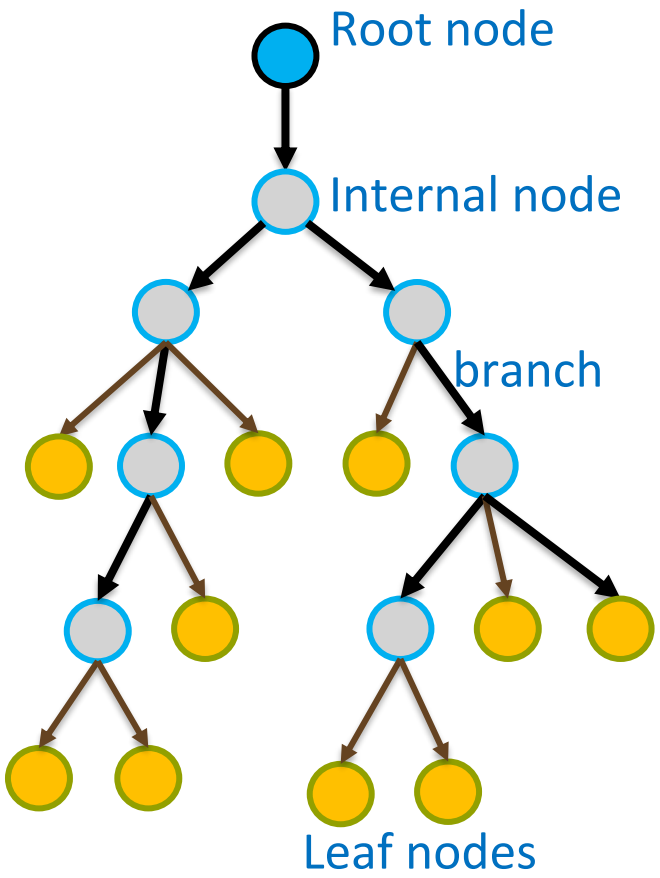
Topic 7: Decision Trees & Their Variants



Contents

Introduction to Decision Trees

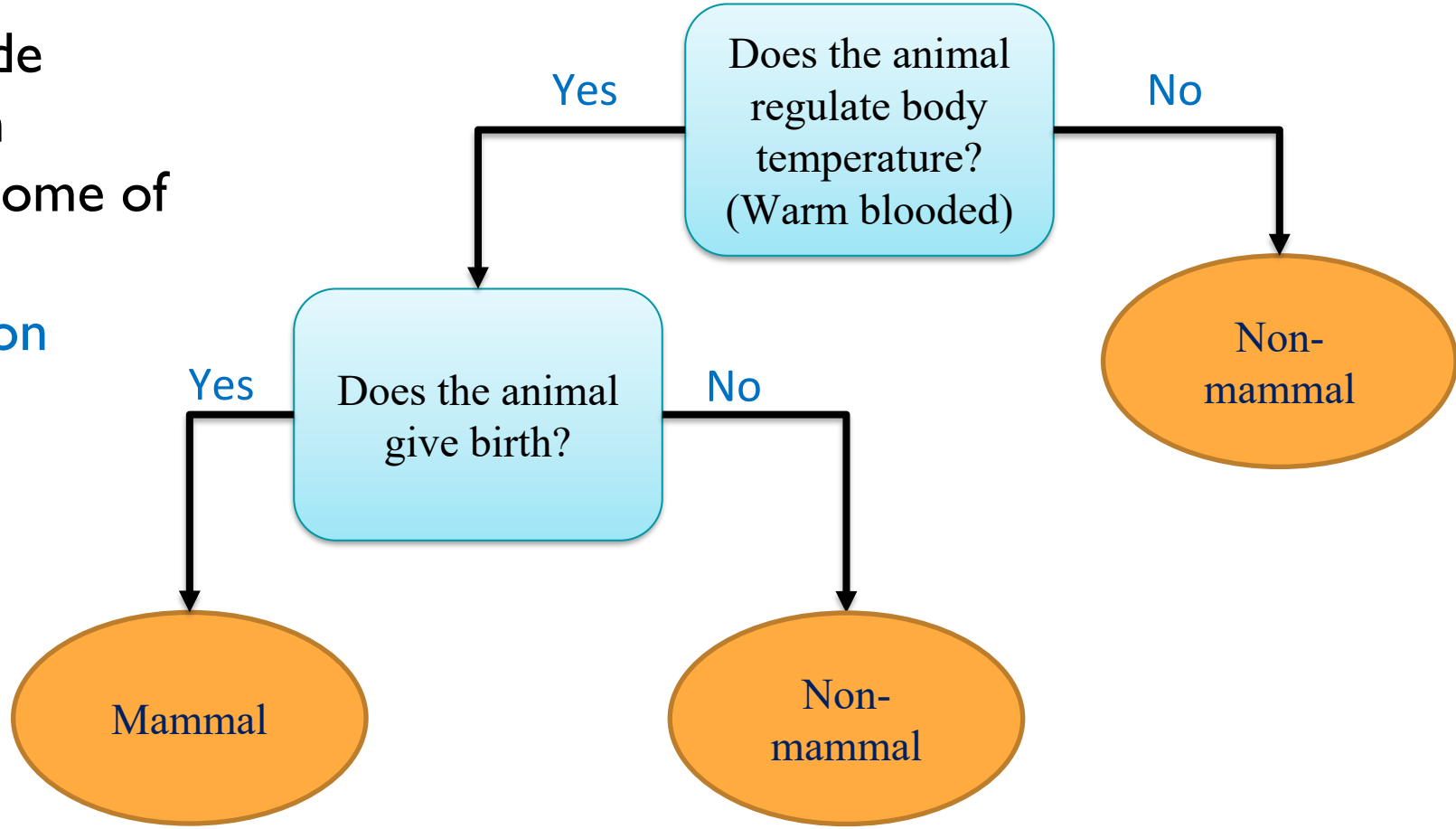
- One of the most popular tools in machine learning used for both classification and regression tasks
- Decision trees are also the basis for other machine learning techniques such as bagging, random forest, boosting, etc.
- Key Idea: A decision tree is a rooted tree that represents a set of prediction rules obtained by recursively partitioning the feature space



A rooted tree

Decision Tree – Intuition & Example

- Intuition: Each root/internal node represents a question/condition
- Each branch represents an outcome of the question
- Each leaf node denotes a decision



A hand-designed decision tree to decide whether an animal is mammal or not

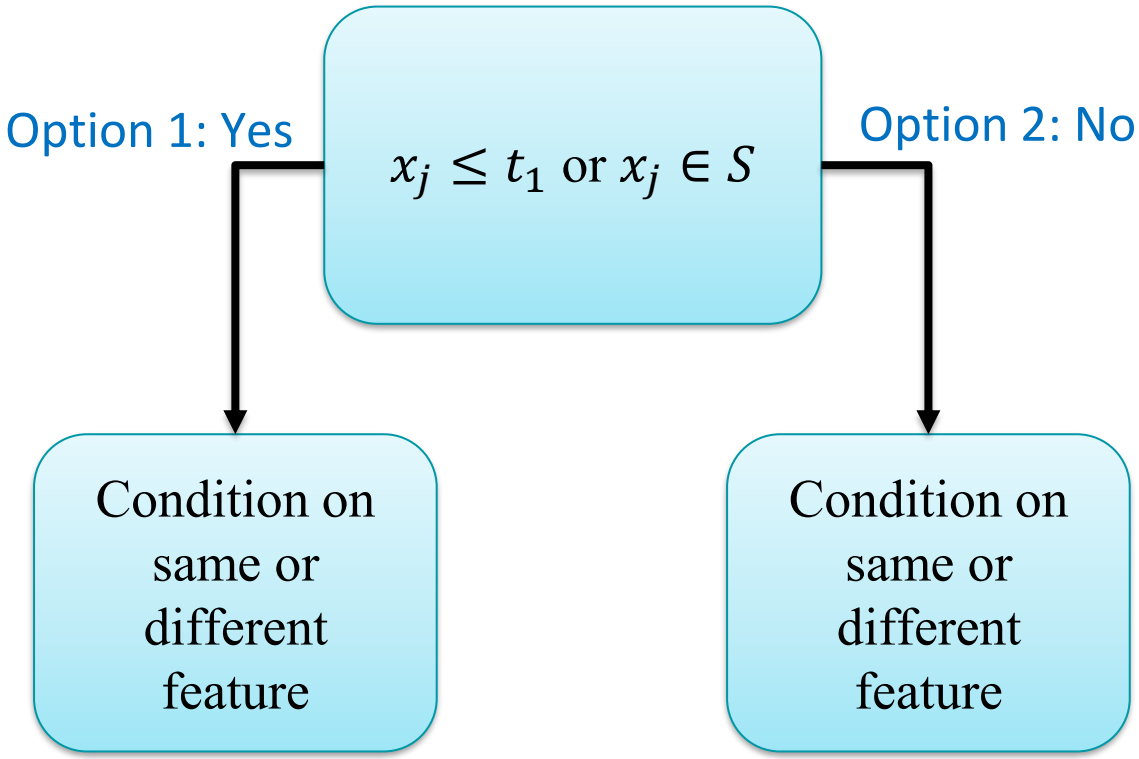
How to use a decision tree to classify data with multiple input features into different target classes?

Decision Tree for Classification

- Outputs: Decision in this case is to decide the class to which sample belongs

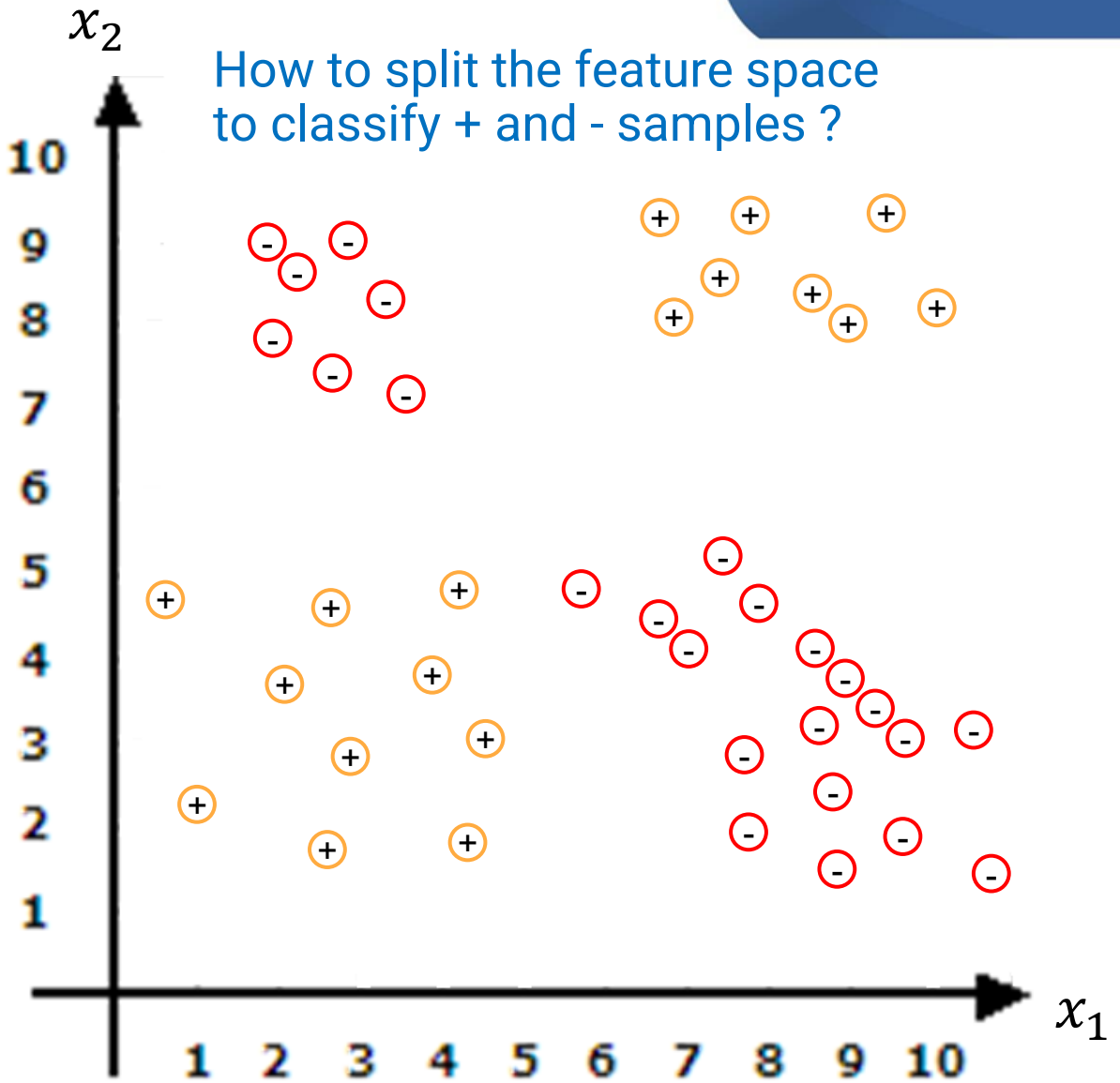
$$y = \{1,2, \dots, m\}$$

- Nodes: Represent a question/condition on an input feature based on which tree splits into branches (conditional split)
- Branches: Represent different outcomes or options that are available based on the nodal condition
- Generally split is binary (yes/no) but multiway splits (discrete outcomes) are also possible

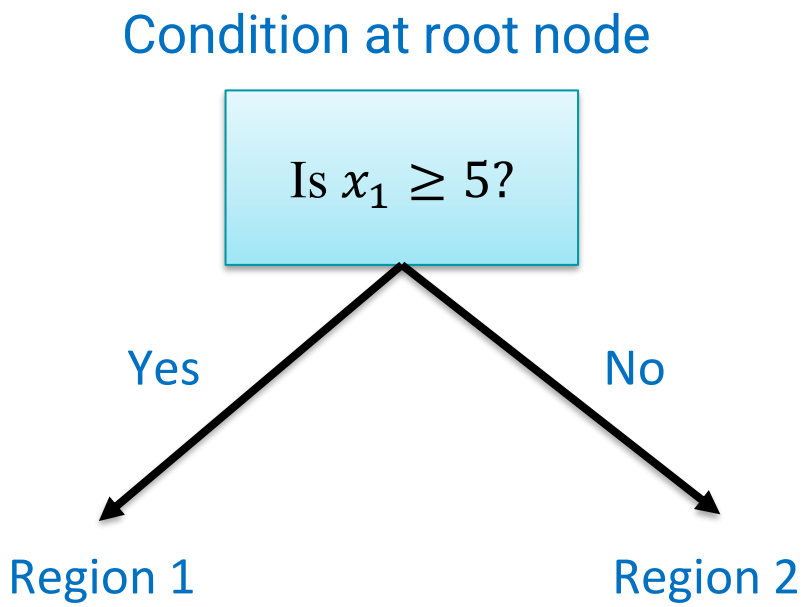


Decision Tree for Classification: Feature Space Perspective

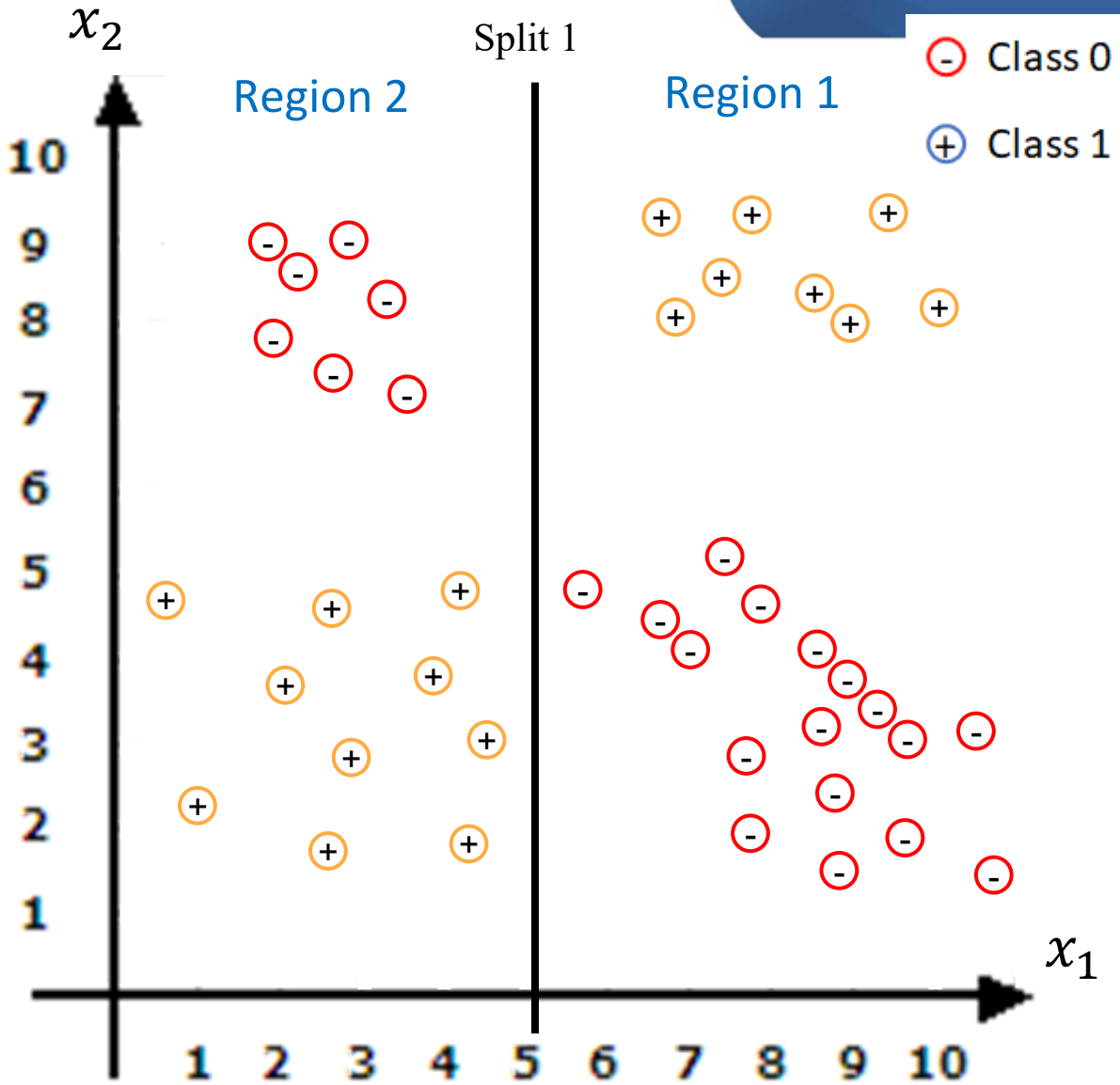
- Input Space: $\mathbb{R}^D$ or mixed numerical/categorical feature space
- Decision tree splits the feature space into multiple regions R_c based on the conditions on features
- Each region should contain samples belonging to one particular class or majority should belong to one class
- Then, each region is assigned a particular class c



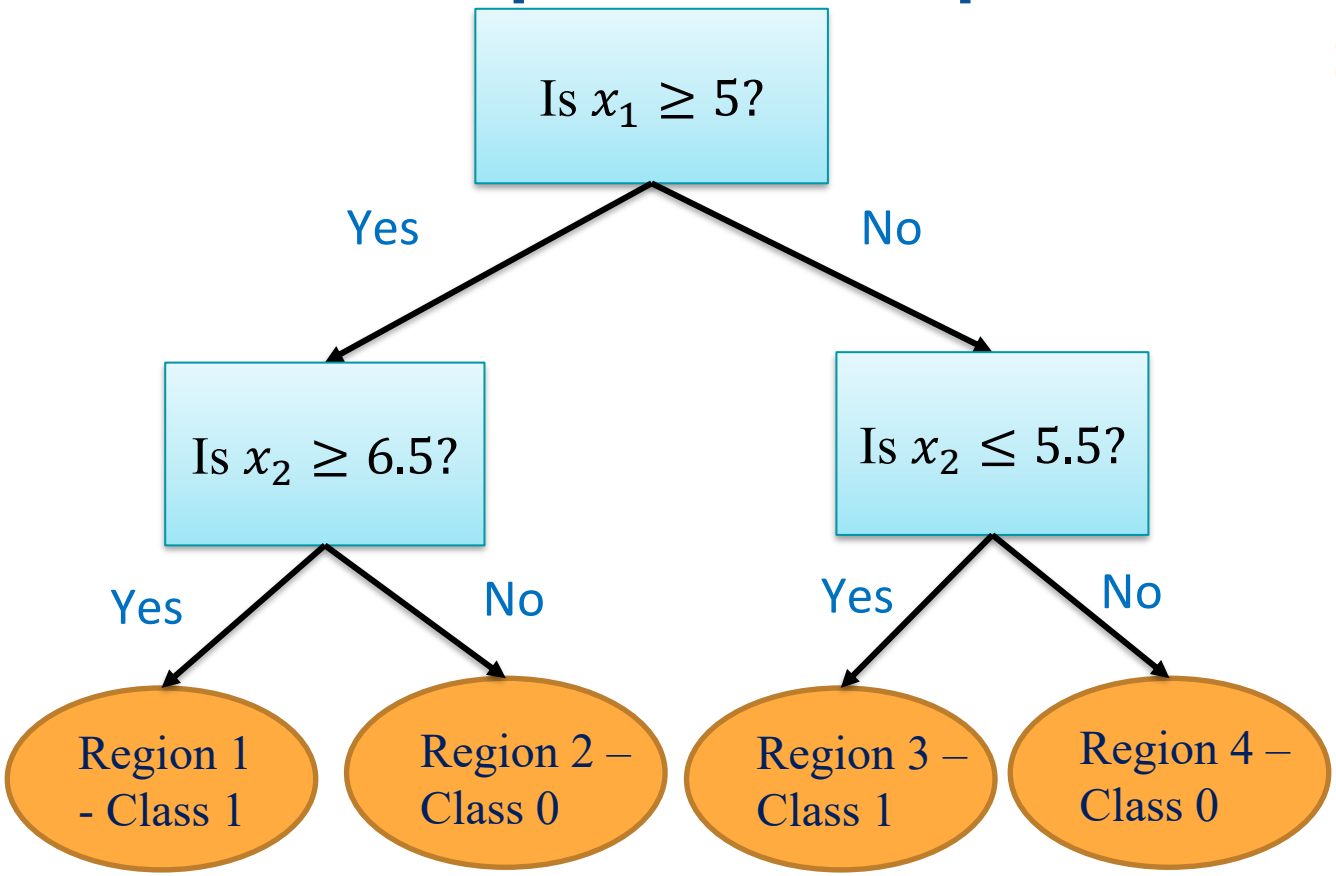
Decision Tree for Classification: Feature Space Perspective

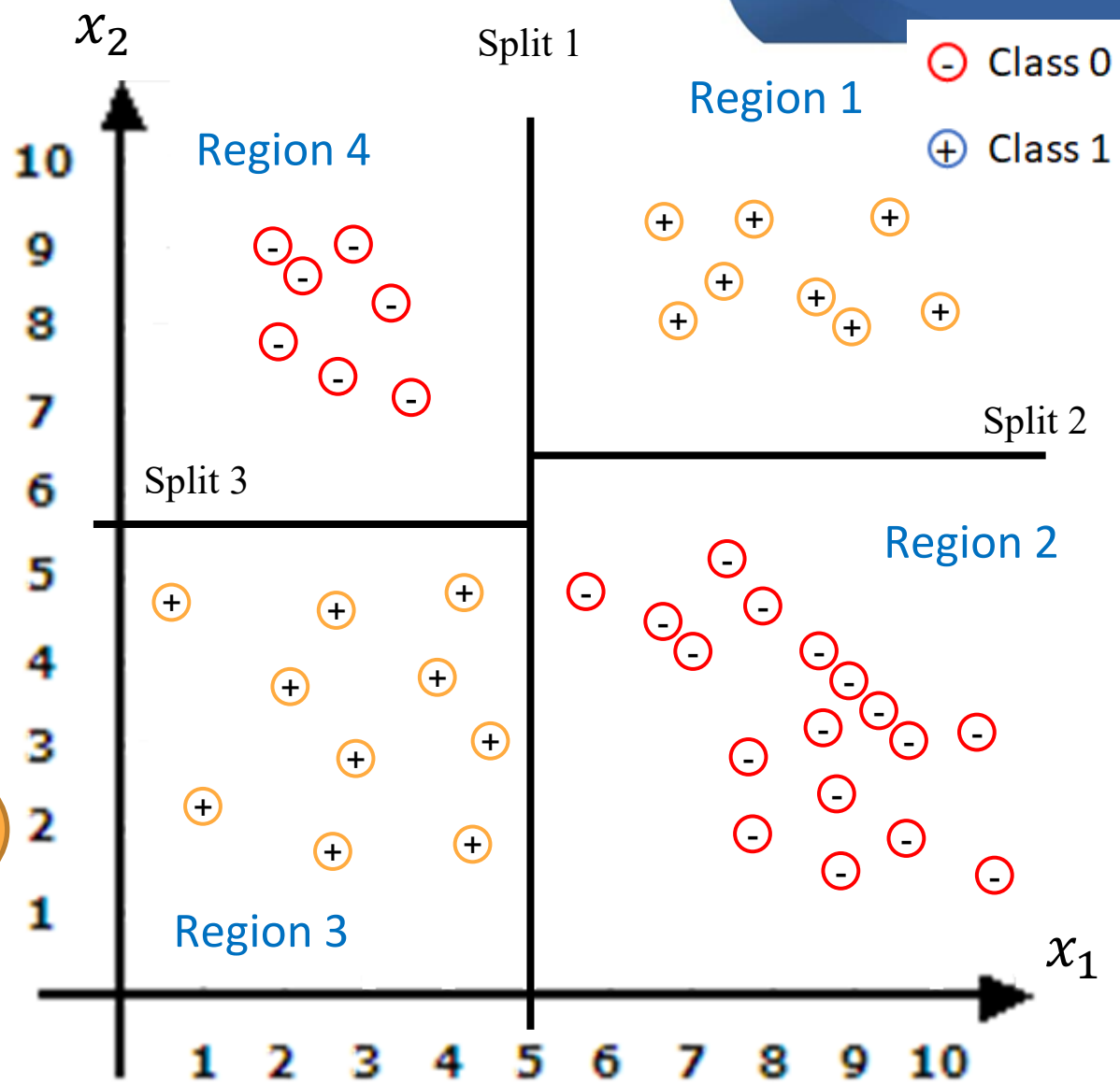


After split 1 at root node, the regions still contain samples of both classes



Decision Tree for Classification: Feature Space Perspective



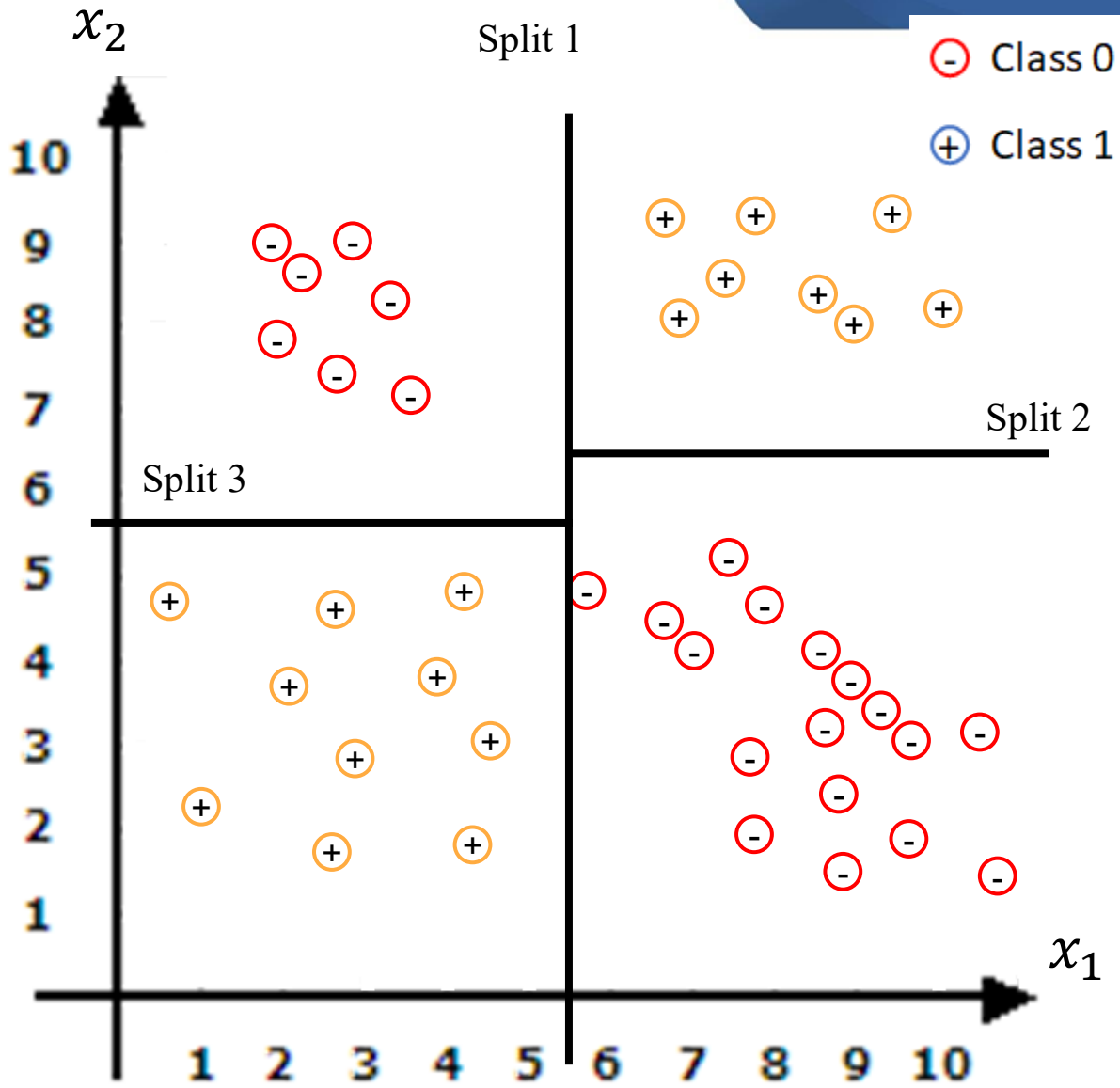


Each leaf node of the decision tree corresponds to a region in the feature space

Building a Decision Tree for Classification

Building a Decision Tree for Classification

- Questions to be answered while building a decision tree:
  - How to select the feature and the condition to split upon at a given node?
  - When to stop splitting or fixing a node to be the leaf node ? What is the stopping criteria?

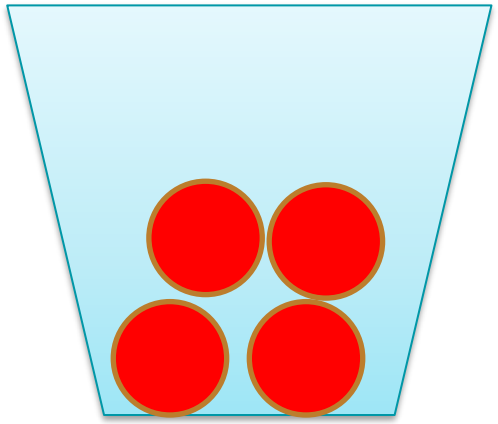


Building a Decision Tree for Classification: Feature and Condition to Split

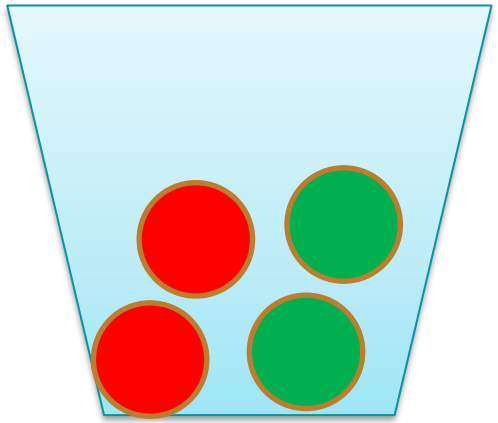
- Feature and condition for splitting can be selected randomly but for an efficient split, entropy or gini index are used to determine the split at a node

Gini Index and Entropy

- Purity: Indicates the homogeneity of a group of items (class of items)
- Example: Suppose there is a jar which can contain red and green balls
- How to quantify this purity mathematically when dealing with samples of different classes? – Gini index and Entropy



Jar 1 with highest purity

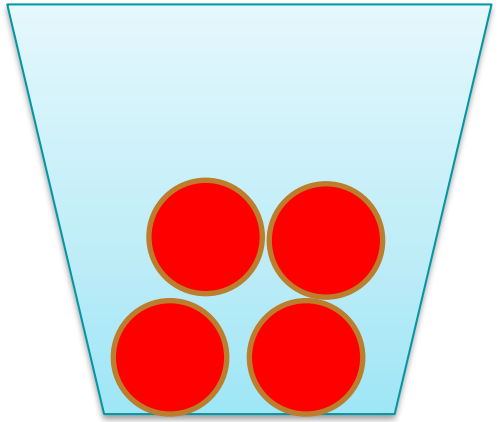


Jar 2 with least purity

Gini Index

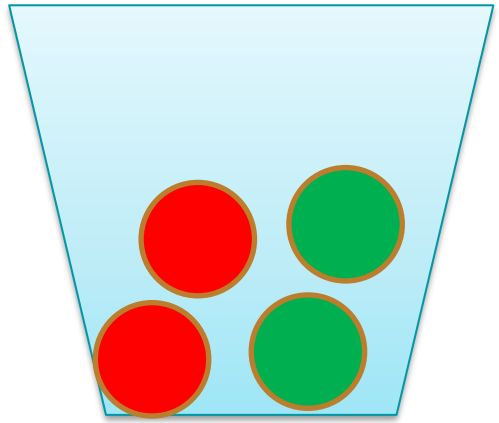
- Gini index: Measure of impurity of a set of samples

$$Gini = 1 - \sum_j p_j^2$$



Jar 1

- Jar 1: $Gini = 1 - \left(p_r^2 + p_g^2\right) = 1 - \left(1^2 + 0^2\right) = 0$
- Jar 2: $Gini = 1 - \left(p_r^2 + p_g^2\right) = 1 - \left(0.5^2 + 0.5^2\right) = 0.5$
- Gini index value of 0 indicates most pure set
- For two classes, the maximum Gini impurity is 0.5, attained when both classes are equally likely
- Interpretation: Probability of misclassification if a label is assigned at random according to the class proportions at that node

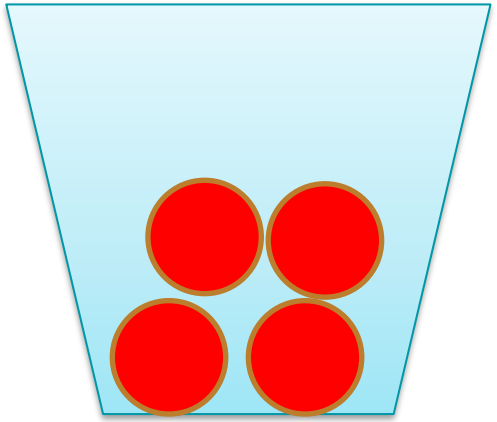


Jar 2

Entropy

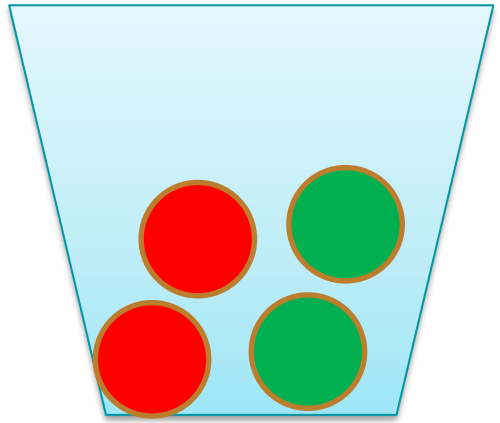
- Entropy: Measure of impurity of a set of samples
- Definition:

$$Entropy = -\sum_j p_j \log_2 p_j$$



Jar 1

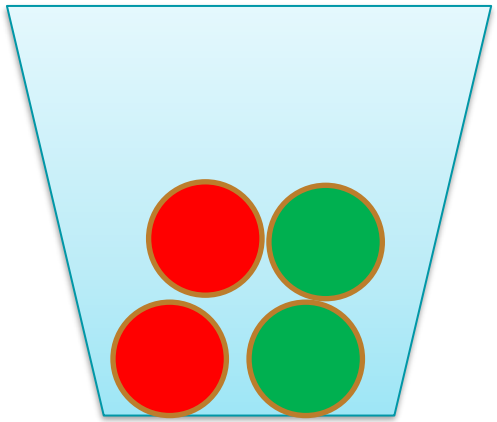
- Jar 1: $Entropy = -(p_r \log p_r + p_g \log p_g) = (0 + 0) = 0$
- Jar 2: $Entropy = -(p_r \log p_r + p_g \log p_g) = -(-0.5 - 0.5) = 1$
- Entropy value of 0 indicates most pure set
- For two classes, the maximum entropy is 1, attained at $p_1 = p_2 = 0.5$



Jar 2

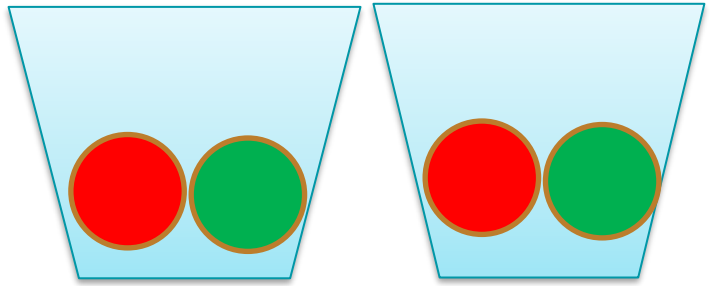
Building a Decision Tree for Classification: Feature and Condition to Split

- For an efficient split, entropy or gini index are used to determine the split at a node
- Question: How to use entropy or gini index to efficiently perform a split?
- Answer: Total Entropy or gini index should reduce after the split at a node



Jar 1

$Gini = 0.5$

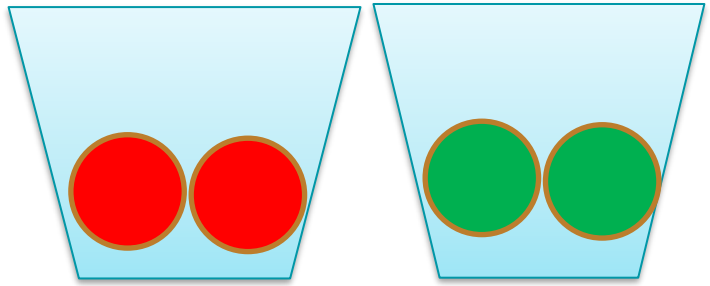


Jar 2a

$Gini = 0.5$

Jar 2b

$Gini = 0.5$



Jar 3a

$Gini = 0$

Jar 3b

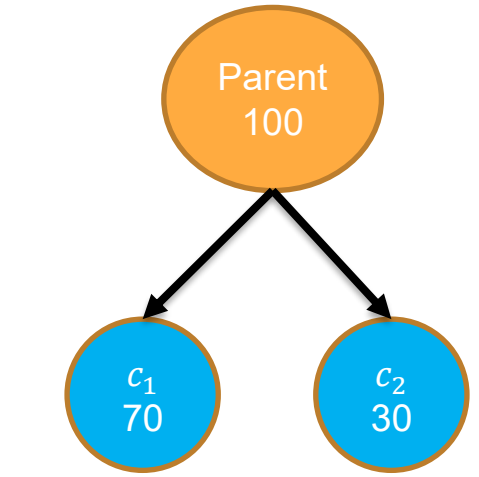
$Gini = 0$

Building a Decision Tree for Classification: Feature and Condition to Split

- Good Split: Total Entropy or gini index reduces after the split at a node
- Information gain or Gini gain is used to quantify this reduction in entropy or gini index
- Information gain (or Gini gain): Change in the entropy or gini index after the split (change in entropy from parent node to child nodes)

$$\Delta i = i(p) - \left(w_{c1} * i(c_1) + w_{c2} * i(c_2)\right)$$

  - w_{c_1}: Fraction of samples in child node c_1
  - w_{c_2}: Fraction of samples in child node c_2



$w_{c_1} = 0.7$ $w_{c_2} = 0.3$

- Objective: To split the node such that the information or Gini gain is maximised

Building a Decision Tree for Classification: Feature and Condition to Split

- Objective: To split the node such that the information gain is maximised

$$\Delta i = i(p) - \left(w_{c1} * i(c_1) + w_{c2} * i(c_2)\right)$$

- Issue: What if the number of possible splits is infinite – Features which take continuous values
- Solutions:
  1. Order the feature values and split at mid value of every pair
  2. Clustering the values in that feature into predefined number groups and taking mid values between cluster centres

$$x_1 = \begin{bmatrix} 1.2 \\ 1.5 \\ 2.4 \\ 2.7 \\ 5.6 \\ 5.9 \end{bmatrix}$$

Possible Splits:

$x_1 \leq 1.35$

$x_1 \leq 2.55$

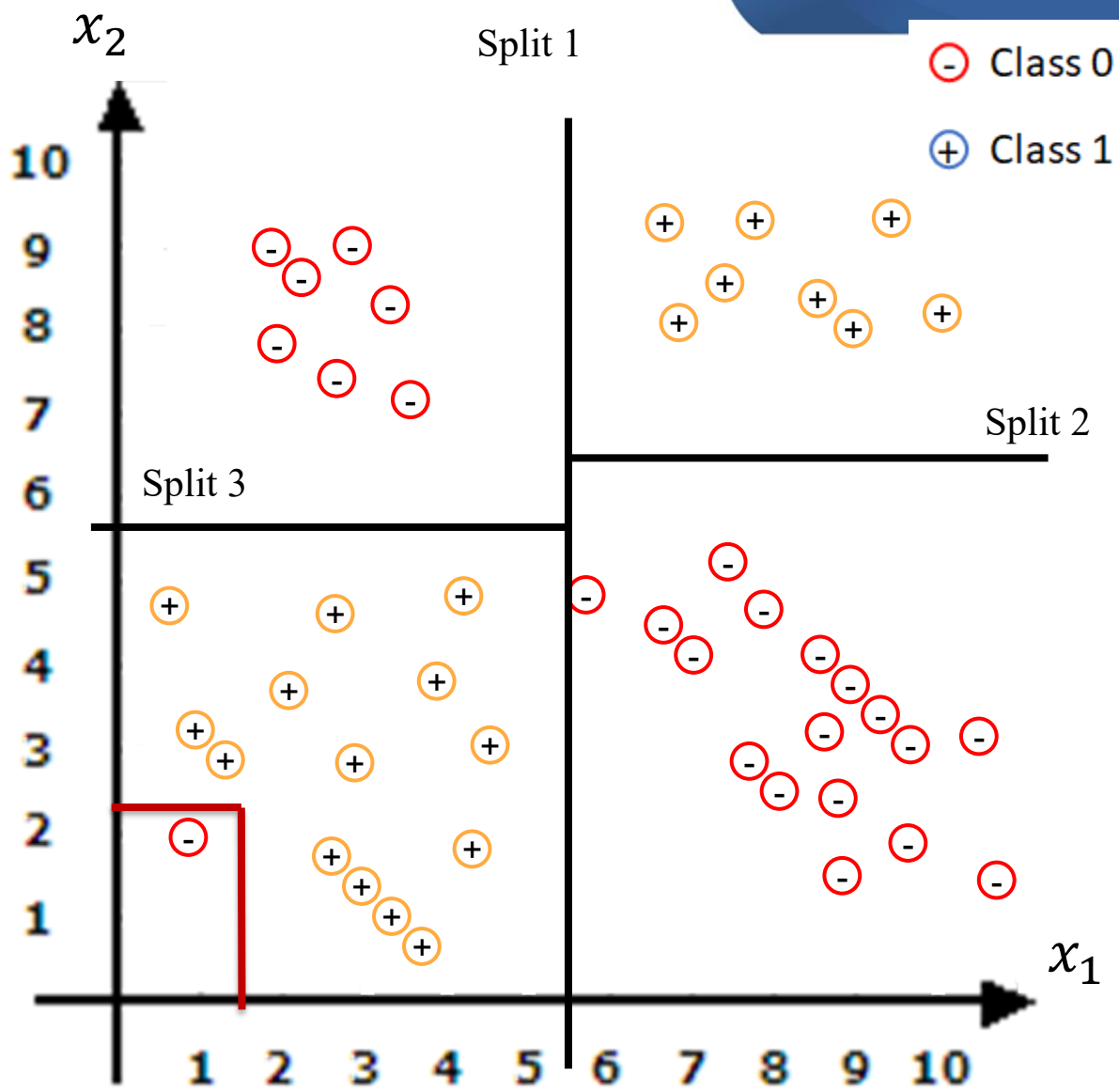
$x_1 \leq 5.75$

Building a Decision Tree for Classification

- Questions to be answered while building a decision tree:
  - How to select the feature and the condition to split upon at a given node?
  - When to stop splitting or fixing a node to be the leaf node ? What is the stopping criteria?

Building a Decision Tree for Classification: Stopping Criteria

- Ideally, splitting should stop when all the samples at a node belong to the same class i.e., the node is a leaf node
- Issue: Lack of generalisation when the leaf node contains very few samples – overfitting of data
- Solutions:
  - Early stopping – Stop building the tree after a predefined criteria is met
  - Pruning – After building a tree, remove the leaf nodes causing overfitting



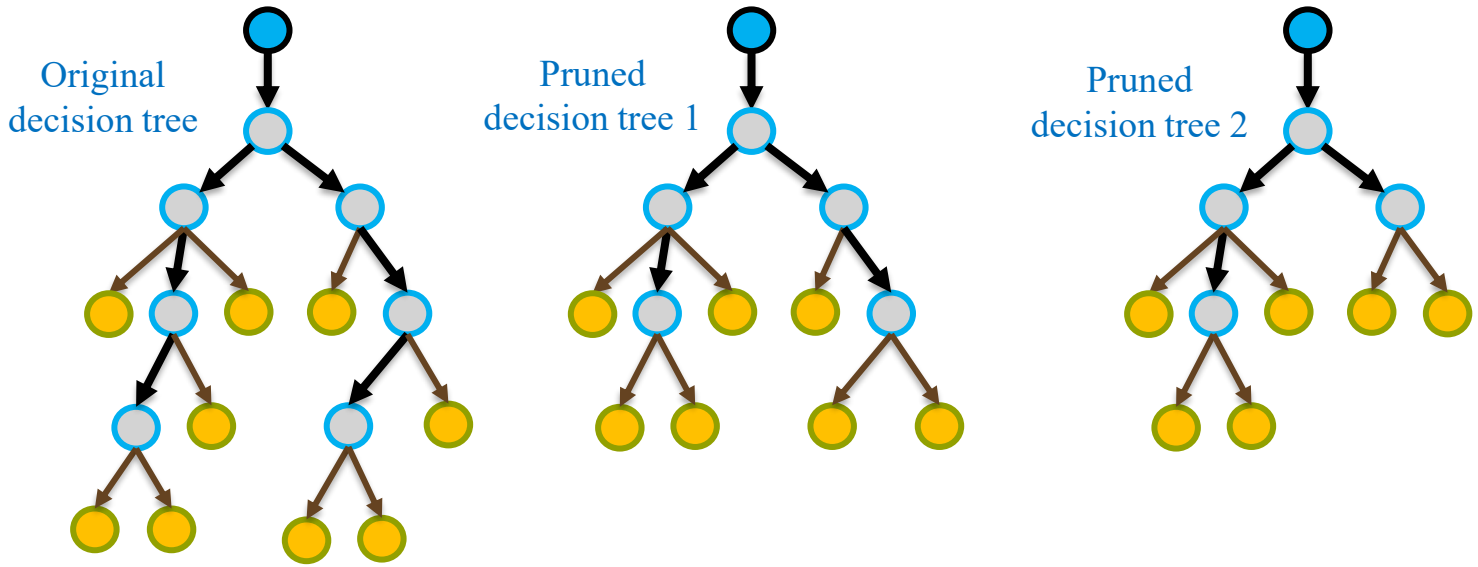
Building a Decision Tree for Classification: Stopping Criteria – Early Stopping

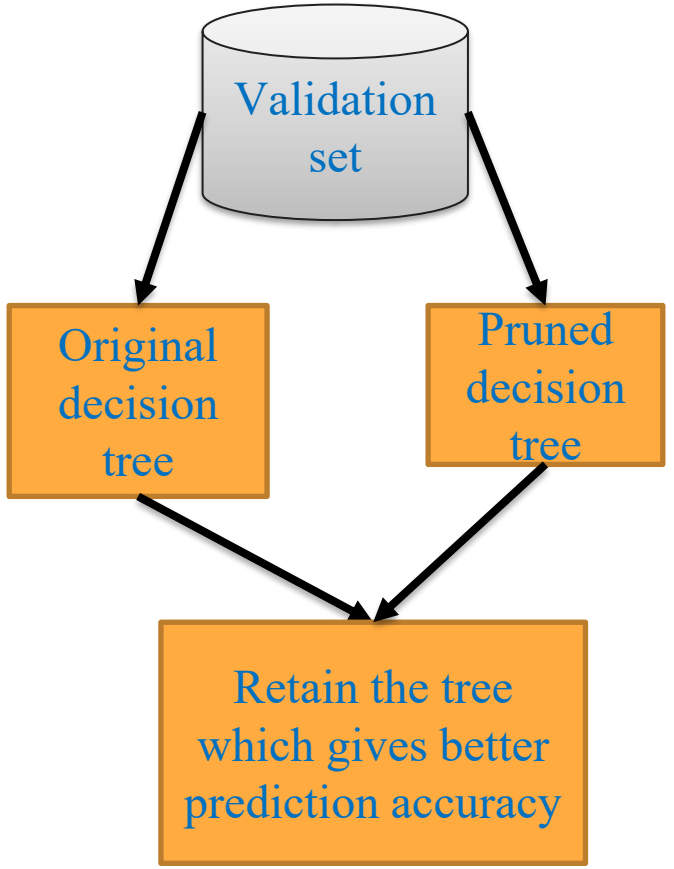
- Following are different early stopping techniques while building decision trees:
  - Maximum tree depth: Maximum number maximum layers for the tree can be pre-defined
  - Minimum samples in a leaf node: Pre-define the minimum number of samples that a leaf node may contain
  - Minimum gain: Pre-define the minimum information or Gini gain at a split below which no more splits are performed at the node

- In practice, all of these are combinedly used as stopping criteria

Building a Decision Tree for Classification: Stopping Criteria – Pruning

- Pruning: Removal of leaf nodes and branches on leaf nodes to reduce complexity of a tree
- Validation set: Used to validate if pruned tree performs better than original decision tree
- Build multiple trees with different nodes pruned and retain the one which performs best on validation set





Building a Decision Tree for Classification: Summary of Steps

- Input: Training data with features values and target class labels
- Step 1: List the possible splits under each feature and calculate the information gain (entropy or gini index) when the split is performed
- Step 2: Split the tree based on the condition which results in maximum information gain – Feature space gets split into two subspaces
- Step 3: Perform further splits again based on maximum information or gini gain – Each of the feature subspaces get split further
- Step 4: Repeat until the stopping criteria is reached
- Step 5: Performing pruning using a validation set

Gini Index Vs Entropy – When to Use?

- Gini Index:
  - Gini index is computationally simple and fast
  - Preferred when computational efficiency is important (for large datasets)
  - Used by default in many libraries
  - Gini often favors splits that quickly increase node purity, especially by isolating a dominant class
- Entropy:
  - Entropy is computationally expensive due to logs
  - Can be slightly more sensitive to changes in class probabilities
  - Sometimes prefers splits that reduce overall class uncertainty more strongly
  - In practice, often gives splits similar to Gini

Eg: Entropy changing more sharply

$$G(p) = 1 - p^2 - (1-p)^2 = 2p(1-p)$$

$$E(p) = -p\log_2 p - (1-p)\log_2(1-p)$$

$p = 0.5$	$G(p) = 0.5$	$E(p) = 1$
$p = 0.8$	$G(p) = 0.32$	$E(p) = 0.722$
$p = 0.95$	$G(p) = 0.095$	$E(p) = 0.286$

Eg: 100 samples → Class A – 90, $P_A = 0.9$

100 samples → Class B – 10, $P_B = 01$

Split: $C_1 = (0A, 9B)$, $G_1 = 0$

$C_2 = (90A, 9B)$, $G_2 = 1 - \left(\frac{90}{99}\right)^2 - \left(\frac{9}{99}\right)^2 = 0.1653$

Wgt Gini $= \frac{1}{100}(0) + \frac{99}{100}(0.1653) = 0.1636$

Entropy: $H_1 = 0$

$H_2 = 0.4395$

Weighted H $= \frac{1}{100}(0) + \frac{99}{100}(0.4395) = 0.4351$

Split 2: C_1 : (19A, 6B)

C_2 : (71A, 4B)

Wgt $G^1 = 0.1636$

Wgt $G^2 = 0.1669$

Gini: $G_1 = 1 - \left(\frac{19}{25}\right)^2 - \left(\frac{6}{25}\right)^2 = 0.3648$

$G_2 = 1 - \left(\frac{71}{75}\right)^2 - \left(\frac{4}{75}\right)^2 = 0.1009$

Entropy: $E_1 = 0.795$

$E_2 = 0.3004$

Wgt $E = \frac{25}{100}(0.795) + \frac{75}{100}(0.8004)$

$E^1 = 0.4351$

$E^2 = 0.424$

Decision Tree Building Algorithms

Classical Decision Tree Algorithms

- Different decision tree algorithms were developed to answer the same core questions in different ways:
  - How should the best split be selected?
  - Should the tree use binary splits or multiway splits?
  - Can the method handle continuous features?
  - Does it support classification only or regression also?
  - How should the tree be pruned to avoid overfitting?

Classical Decision Tree Algorithms

ID3 — Iterative Dichotomiser 3

- Proposed by J. Ross Quinlan
- One of the earliest and most influential tree-learning algorithms
- Designed mainly for classification
- Uses Information Gain for split selection
- Allows multiway splits at categoricals

C4.5

- Proposed by Ross Quinlan
- Developed as an improvement over ID3
- Still mainly for classification
- Uses Gain Ratio, a normalized version of Information Gain; normalized by how fragmented is the split
- Added important practical improvements such as handling continuous attributes, and pruning
- Allows multiway splits at categoricals

CART — Classification and Regression Trees

- Proposed by Breiman, Friedman, Olshen, and Stone
- A broader framework than ID3/C4.5
- Supports both classification and regression
- Uses Gini index for classification and squared-error / variance reduction for regression
- Binary splits only
- Forms the basis of many modern tree implementations

Decision Tree Tuning

- Goal of tuning: Choose hyperparameters that control tree complexity and improve test performance
- Common tuning parameters
  - Maximum depth of the tree
  - Minimum samples to split a node
  - Minimum samples in a leaf
  - Minimum impurity decrease / minimum gain
  - Split criterion: Gini vs Entropy for classification

Advantages and Disadvantages of Decision Trees

Advantages:

- Easy to interpret and explain
- They do not require any domain knowledge for feature selection
- Can easily handle both numeric and categorical data
- Does not require any scaling
- The type of relation (linear or non-linear) between features and targets does not affect tree performance

Disadvantages:

- Decision tree learners can create complex trees that overfit the data and do not generalise well to unseen data
- Trees can be non-robust i.e., a small change in the data can cause a large change in the tree built

Regression Trees

Regression Trees

- Decisions trees are rarely used for regression tasks
- Differences of a regression tree from a classification tree:
  1. Measure of impurity: Choose the split that minimises the sum of squared errors with respect to the mean target value in the child nodes
     - Suppose the feature space is split into two regions (corresponding to two child nodes): r_1 and r_2
     - Mean target value in r_1: $\overline{y_{r_1}} = \sum_{i \in r_1} y_i \,/|r_1|$
     - Mean target value in r_2: $\overline{y_{r_2}} = \sum_{i \in r_2} y_i \,/|r_2|$
     - Sum of squared errors wr.t. mean target:

$$e_{r_1} + e_{r_2} = \sum_{i \in r_1} \left(y_i - \overline{y_{r_1}}\right)^2 + \sum_{i \in r_2} \left(y_i - \overline{y_{r_2}}\right)^2$$

     - Choose split which : $\min e_{r_1} + e_{r_2}$

Eg: Samples in region r_1:

#	Target (y)
1	10
2	15
3	20
4	11

$$\overline{y_{r_1}} = \frac{56}{4} = 14$$

$$e_{r_1} = (10-14)^2 + (15-14)^2 + (20-14)^2 + (11-14)^2$$

$$e_{r_1} = 62$$

Regression Trees

- Regression trees are mostly used in popular ensemble methods
- Differences of a regression tree from a classification tree:
  1. Measure of impurity: Choose the split that minimises the sum of squared errors with respect to the mean target value in the child nodes
  2. Predictions: At a leaf node, the predicted value for a test sample is equal the mean of the target values of the samples present at that leaf node
  - Suppose r_l is the region corresponding to leaf node
  - Mean target value in r_l: $\overline{y_{r_l}} = \sum_{i \in r_l} y_i \,/|r_l|$
- Stopping criteria can be like the ones used in classification trees

Ensemble of Decision Trees

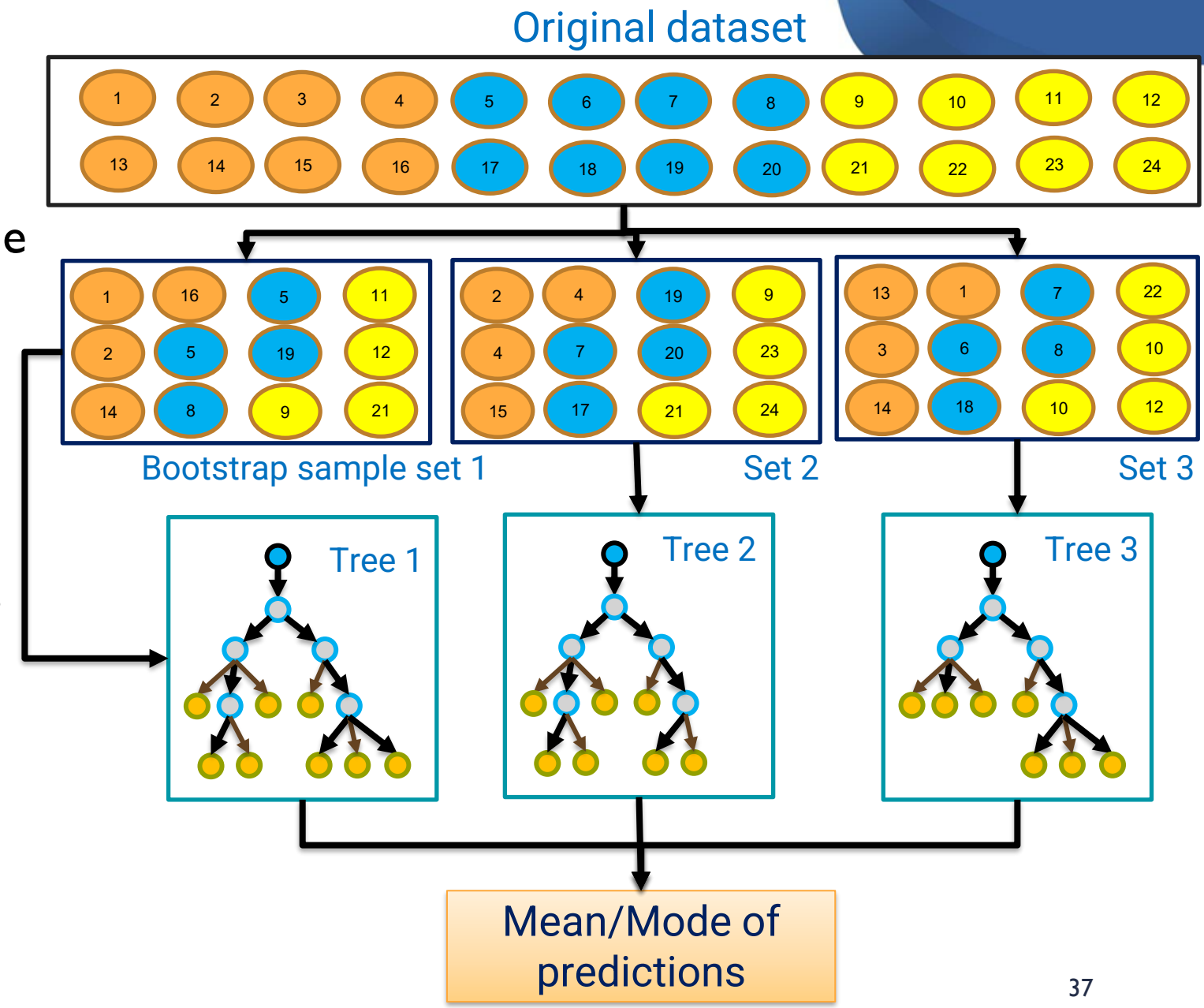
Why Ensemble Decision Trees?

- A single decision tree is:
  - highly sensitive to training data
  - unstable (small data changes can produce very different trees)
  - prone to overfitting, especially when deep
- Idea: Combine several decision trees to get better predictive performance – ensemble of decision trees
  - prediction variance can reduce
  - generalization improves
  - accuracy becomes much higher
- Ensemble Strategies:
  - Parallel combination: Build many independent trees to reduce variance – Bagging & Random Forest
  - Sequential combination: Build trees one after another with improvement to reduce bias - Boosting

Tree Bagging

- Steps in Tree Bagging:
  1. Create multiple datasets from the given dataset by sampling data points with replacement - 'bootstraping'
  2. Fit a decision tree to each re-sampled dataset
  3. On a test set, mean/mode of the predictions of all the trees is taken to make a prediction

➢ Bagging introduces randomness into samples of data



Why Does Bagging Improve Decision Trees?

- Bagging reduces prediction variance by averaging multiple trees

B trees: $\hat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B} f_b(x)$

- Variance of ensemble prediction is given by:

$Var(\hat{f}_{bag}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$

σ^2: Variance of a tree prediction

ρ: pairwise correlation of tree predictions

- Implication if trees are not perfectly correlated:

$\rho < 1 \Rightarrow Var(\hat{f}_{bag}) \leq \sigma^2$

- Bagging improves stability because:
  - decision trees are high-variance learners
  - bootstrap samples create multiple slightly different trees
  - averaging reduces fluctuations

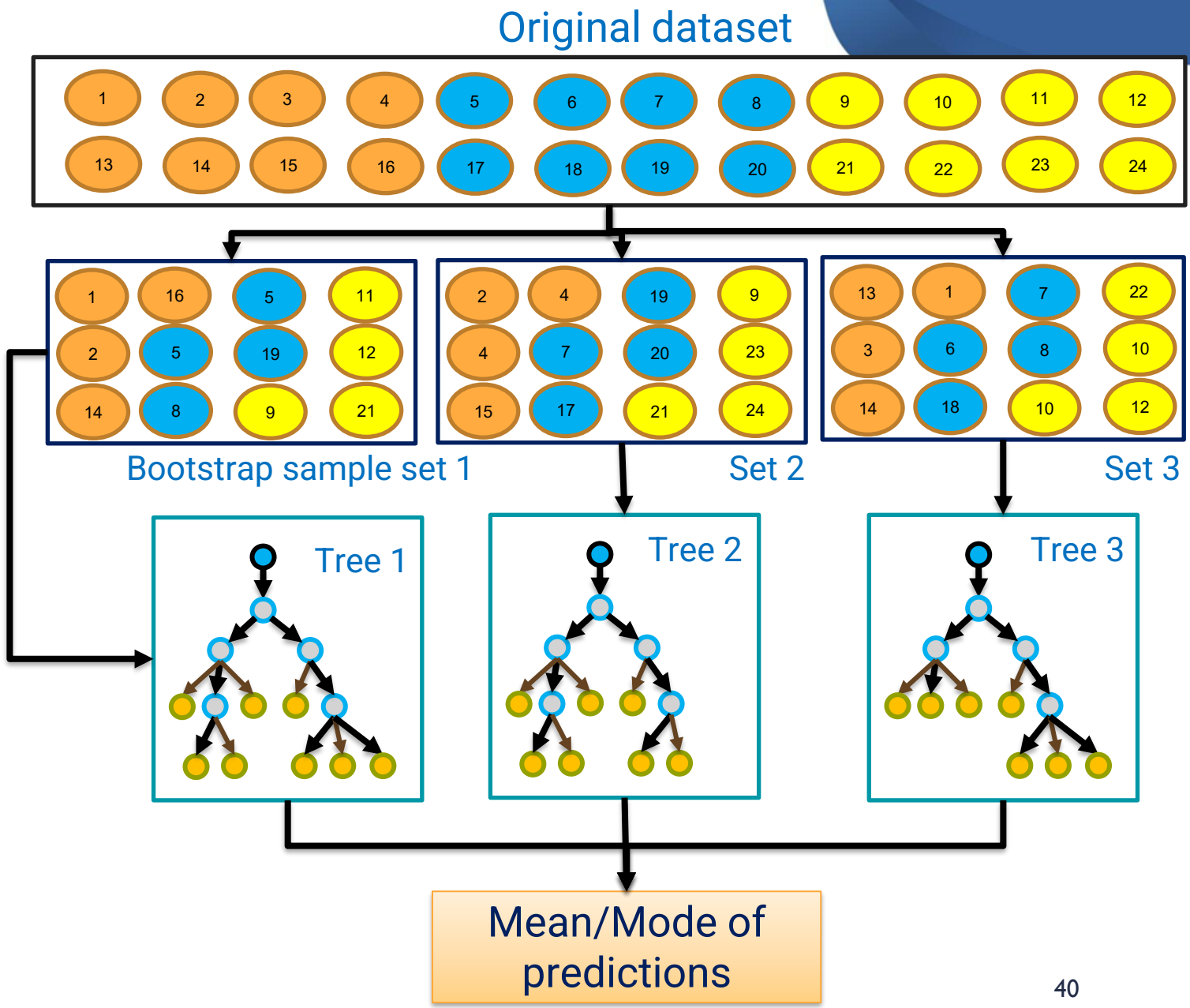
Motivation for Random Forest

- Limitations of Bagging:
- Even after bootstrap sampling:
  - strong input features often dominate many trees
  - many trees become structurally similar
  - tree correlation (ρ) remains high
- Hence variance reduction is limited

- Q: How to reduce variance further?
- A: Introduce randomness in feature selection as well which reduces correlation between trees

Random Forest

- Ensemble of trees which are built in a similar manner to bagging
- However, at each node random subset of D features is sampled, and the best split is chosen only among those features
- Each split in Trees 1, 2 and 3 is based on gain observed at m features only
- Random forests introduce randomness into both samples and features
- Hence, the trees are more diverse and we get better predictions



Questions in Building Random Forest

- How many number of decision trees to create?
  - Sample datasets & build trees until the validation error no longer decreases
- How to choose the value of r (number of features)?
  - Typical defaults:
    - Classification $r = \sqrt{D}$
    - Regression $r = \frac{D}{3}$
- How deep to go?
  - Random forests usually go deeper than individual trees and pruning is not done as overfitting is handled by the ensemble

Intro to Tree Boosting

Motivation for Boosting

- Issue with Random Forest:
  - Training of all trees happens in parallel
  - If many trees struggle with certain difficult patterns, there's no mechanism to correct their mistakes.
  - The final prediction is based on majority vote or averaging, but persistent errors across trees remain uncorrected.
- Question: Can we train learners sequentially to focus on mistakes of previous learners?
- Idea of Boosting: Combines many weak learners sequentially to produce a stronger model
- Learners could be small decision trees, linear classifiers, logistic regression variants, etc.

Boosting Analogy

- Simple Analogy for Boosting:
  - Suppose you are learning to play a song on a piano:
  - On your first attempt, you make mistakes.
  - The second time, you focus more on the notes you got wrong.
  - Each time you practice, you pay more attention to the parts you messed up earlier, and gradually the whole performance improves.

Boosting as an Additive Model

- Boosting builds the final predictor by adding one learner at a time (additive model)

$$F_M(x) := \sum_{m=1}^{M} \eta_m f_m(x)$$

$f_m(x)$: o/p of a learner added at step m

η_m: weight associated with learner m

$F_M(x)$: Ensemble model o/p

Recursive eq: $F_m(x) = F_{m-1}(x) + \eta_m f_m(x)$

- Boosting is primarily aimed at improving the model stage by stage, often leading to bias reduction

Tree Boosting Algorithms

- Tree Boosting = Additive model with trees
- $f_m(x)$: Small decision tree

- Different tree boosting methods differ in how they choose:
  - the next tree
  - its weight
  - the objective/loss function

- Different Boosting Algorithms:
  - AdaBoost: Increases focus on misclassified points
  - Gradient Boosting: Fits the negative gradient or pseudo-residusla
  - XgBoost, LightGBM, CatBoost: Optimized versions of Gradient Boosting

AdaBoost

Adaptive Boosting

- AdaBoost builds a classifier sequentially by giving more weight to samples misclassified by the previous tree
- Training Idea:
  - At stage m, choose a tree to minimize the weighted classification error:

$$\varepsilon_m = \frac{\sum_{i=1}^{N} w_i \cdot 1(y_i \neq f_m(x_i))}{\sum_{i=1}^{N} w_i} \rightarrow \text{Indicator fn}$$

- Weight update rule:
  - If a sample is misclassified, increase its weight
  - If correctly classified, decrease its weight
  - Exponential update is performed to achieve this
- Learner weight (derived from exponential loss function):

$$\eta_m = \frac{1}{2} \ln\left(\frac{1-\varepsilon_m}{\varepsilon_m}\right)$$

$\varepsilon_m = 0.4 \rightarrow \eta_m \downarrow$

$\varepsilon_m = 0.1 \rightarrow \eta_m \uparrow$

- Smaller error implies larger weight i.e., better leaners get more weightage

Adaptive Boosting: Steps

Assume binary classification with labels +1,-1

Step 1: Initialise all sample weights as $w_i^{(0)} = \frac{1}{N}$ i.e., all samples are equally important

Step 2: Fit a weak classifier (small decision tree) using weighted training data. Objective is to minimise the weighted classifier error ϵ_m

Step 3: Compute the learner weight η_m

Step 4: Update sample weights as: $w_i^{(m+1)} = w_i^{(m)} \exp(-\eta_m y_i f_m(x_i))$

Normalise weights and repeat steps 2 to 4

Prediction:

$\hat{y}(x) = sign\ (F_M)$

Gradient Boosting

Gradient Boosting

- Gradient boosting is also an additive model:

$$F_M(x) = F_{M-1}(x) + \eta f_m(x)$$

- Learning rate is generally kept constant at all iterations
- Idea of Gradient Boosting:
- Pseudo residuals are computed after each adding each tree (each boosting iteration)
- They tell for each sample, in what direction and by how much should the current prediction change to reduce the loss?
- Pseudo residual is positive ⇒ increase prediction and vice versa
- Pseudo residual high ⇒ higher correction is needed

Gradient Boosting

- For each sample i, the pseudo residual $r_i^{(m)}$ at iteration m is given by:

$$r_i^{(m)} = -\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\Bigg|_{F=F_{m-1}}$$

- For case of regression with squared error loss, the pseudo residual becomes:

$$L = \frac{1}{2}\left(y_i - F_{m-1}(x_i)\right)^2$$

$$r_i^{(m)} = y_i - F_{m-1}(x_i)$$

Gradient Boosting Steps

Start with an initial prediction for all samples, mean (for regression) or log-odds $\log\left(\frac{p}{1-p}\right)$(for classification). p Is fraction of class 1 samples

For each sample, compute the pseudo residual

Train a simple decision tree fitting the pseudo-residuals i.e., each tree is regressing the pseudo residuals

y (Actual output)	F_0 (Initial prediction)	$y - F_0$
10	11	-1
15	14	1
16	18	-2
20	15	5
25	21	4

Gradient Boosting Steps

Start with an initial prediction for all samples, mean (for regression) or log-odds $\log\left(\frac{p}{1-p}\right)$(for classification). p Is fraction of class 1 samples

For each sample, compute the error (pseudo-residual) between the true value and the current prediction

Train a simple decision tree fitting the pseudo-residuals (or gradients). This tree learns to reduce the error in the previous prediction.

Add the predictions of the latest tree to the current prediction, scaled by a learning rate parameter (η) to control overfitting.

Keep adding trees to the ensemble until pseudo-residuals are close to 0 or there is no improvement

What happens: Each iteration attempts to "boost" the performance of the ensemble by addressing the previous errors.

Shallow Trees in Gradient Boosting

- Usually shallow trees (depth 1 to 3) are preferred in gradient boosting
- Because each tree should correct only the pseudo-residual
- If trees are too deep:
  - Overfitting occurs quickly
  - Later trees memorise noise
- Therefore, each tree makes small improvements to build a strong model

XGBoost

Gradient Boosting to XGBoost

- Classical gradient boosting works well but has practical limitations:
  - split selection uses only first-order gradient information
  - trees can overfit without explicit regularization
  - training becomes slow for large datasets
  - missing values need preprocessing in standard implementations
- Q: Can we redesign gradient boosting to be mathematically robust and computationally faster?
- A: XGBoost

XGBoost

- An efficient and scalable version of gradient boosting
- Popular in many fields and industry applications
- Differences from Gradient Boosting:
  - XGBoost adds **regularization on tree complexity** to reduce overfitting
  - XGBoost uses both the gradient and the Hessian (second derivative) of the loss function, for determining more accurate splits i.e., Hessian indicates how good the gradient value is
  - XGBoost is optimized for speed and scalability compared to gradient boosting
    - parallel split search
    - sparsity-aware learning
    - cache optimization
  - Handles missing values in the data natively without the need for imputation

Efficient Engineering in XGBoost

- XGBoost is designed not only to improve prediction quality, but also to train efficiently on large datasets.
- Parallel Split Search
  - For each feature, possible split points are evaluated independently.
  - These computations can run in parallel across CPU or GPU cores.
  - This speeds up tree construction significantly compared to standard gradient boosting.
- Sparsity-Aware Learning
  - XGBoost efficiently handles sparse data (missing values)
  - During split evaluation, missing entries are skipped and an optimal default direction is learned
  - Useful for real-world datasets with incomplete or sparse features
- Cache Optimization
  - Feature values are stored in sorted block structures for faster repeated access
  - This reduces computation time during split evaluation

XGBoost Handles Missing Values

- When XGBoost evaluates a candidate split (say $x_1 < t$), the samples fall into 3 groups:
  - **Left child node (Yes branch)**: samples with $x_1 < t$
  - **Right child node (No branch)**: samples with $x_1 \geq t$
  - **Missing group**: samples where that feature is missing
- XGBoost temporarily tries these options:
  - Send all missing samples to the left node ⟶ compute loss reduction
  - Send all missing samples to the right node ⟶ compute loss reduction again.
- Whichever direction gives better loss reduction, it becomes default for missing values
- Therefore, XGBoost does not require full data and can handle missing values without the need for imputation

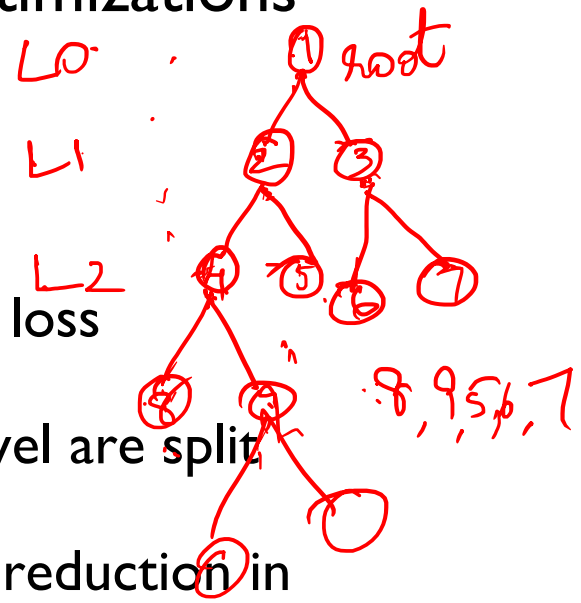
XGBoost for Time Series Forecasting

- XGBoost does not naturally understand time series data but is still found to be very effective for time series forecasting
- Idea: Formulate time series forecasting problem as a supervised learning problem
- Following features can be considered as inputs to the SL problem:
  - Past k values of the time series
  - Rolling averages, seasonal indicators, etc.
  - Day of the week, month, holiday flag, etc.
- Advantages:
  - Handles non-linear trends in time series & external inputs
  - Supports missing values and irregular time intervals

LightGBM

XGBoost to LightGBM

- Light Gradient Boosting Machine (LightGBM) developed by Microsoft Research
- Core idea: Like XGBoost, LightGBM is a gradient-boosted tree method, but it is designed with more aggressive speed and memory optimizations
- Innovations in LightGBM:
  - Histogram-based splitting
    - Continuous features are bucketed into bins
    - Tests split points only at bin boundaries
    - Speeds up training and reduces memory usage for small accuracy loss
  - Leaf-Wise Growth
    - In general, decision trees grow level-by-level i.e., all leaves at a level are split first to ensure balance
    - LightGBM grows by always splitting the leaf that gives the largest reduction in loss, irrespective of the level
    - Reduces training loss faster
    - Trees are deeper and more accurate but can overfit (uses max_depth constraint)



XGBoost to LightGBM

- Innovations in LightGBM:
  - Natively handles Categorical Data
    - LightGBM has stronger native support for categorical splits, reducing the need for one-hot encoding (even available in later versions of XGBoost)
    - Groups categories based on what they majorly tend to predict and reduces the number of branches at a categorical
    - Eg: Colour feature with categories Red, Blue, Green
    - Red and green can be grouped here
    - Adv: Handles high cardinality categoricals easily

Color	Output Label
Red	1
Red	1
Blue	0
Blue	0
Green	1
Green	1

XGBoost Vs LightGBM

Dimension	XGBoost	LightGBM
Speed	Fast (parallel CPU + GPU support), but level-wise tree growth makes it slower on very large datasets	Faster (histogram binning + leaf-wise growth + feature bundling), fewer trees needed
Memory Usage	Higher (stores gradients & Hessians for all data)	Lower (uses histogram binning + compressed features)
Accuracy	Very robust & stable, less risk of overfitting (balanced trees)	Often slightly higher on large data, but can overfit small datasets
Handling Missing Values	Native — learns a default path for missing values during training	Native — treats missing as a separate bin
Handling Categorical Features	Efficient in latest versions but slower than LightGBM	Very efficient — category grouping (no one-hot encoding)
When to Use	Choose when stability & interpretability are important; strong default choice	Choose when dataset is very large, high-dimensional, or categorical-heavy; Suitable for edge deployments

CatBoost

CatBoost: Gradient Boosting for Categoricals

- When categorical variables contain many categories, one-hot encoding may become inefficient
- LightGBM uses direct grouping during split search
  - It is computationally efficient
  - Done for split gain improvement
  - However, it only helps at the split stage at a node
  - Does not create a robust numerical representation of categories that can be re-used across the model (bypasses encoding problem)
- CatBoost: Similar to XBGoost and LightGBM, except that it performs numerical encoding of categorical variables
- Improves performance of gradient boosting when datasets contain many categorical variables
- Core Idea: CatBoost handles categorical variables internally using ordered target statistics

Issue with Rare Categories in Grouping

- Rare categories can look very important just because of accidental small-sample behavior
- Consider this sample set:
  - From data it looks like:
    - Grocery – mostly 0
    - Electronics – mostly 1
    - LuxuryRate – mostly 1
  - Grouping: {Electronics, LuxuryRate} Vs {Grocery}
  - If there is a new sample where LuxuryRate has target 0, the behaviour of the category value suddenly changes from "always 1" to "mixed"
  - So, this leads to a stability issue in split based grouping

Merchant Type	Target
Grocery	0
Grocery	0
Grocery	1
Electronics	1
Electronics	1
Electronics	1
LuxuryRare	1

Target Encoding

- Target encoding is where categorical variables are converted to numerical values which will be used to decide splits
- Idea: Replace each category by its average target
- For the given sample set:
  - For category "Grocery": Target value = 1/3 = 0.33
  - For category "Electronics", Target value = 2/2 = 1
- This is naive target encoding
- Issue with naive target encoding:
  - When predicting target, the model sees a feature value partly computed using the target we are trying to predict
  - Referred to as label leak

Merchant Type	Target
Grocery	0
Grocery	0
Grocery	1
Electronics	1
Electronics	1

Merchant Type	Encoded value	Target
Grocery	0.33	0
Grocery	0.33	0
Grocery	0.33	1
Electronics	1	1
Electronics	1	1

Why Label Leakage is Worse in Boosting

- Naive target encoding already gives slightly optimistic category signals
- First boosting tree learns this optimistic signal
  - For example, category "Electronics" has encoded value 1 and it predicts 1
  - So this category appears highly predictive
- This increases the bias of the model
- Later boosting stages inherit and reinforce this bias
  - Residuals/gradients are computed using current biased predictions
  - Later trees are fit on residuals generated by earlier biased predictions

Merchant Type	Encoded value	Target
Grocery	0.33	0
Grocery	0.33	0
Grocery	0.33	1
Electronics	1	1
Electronics	1	1

$$F_M(x) = F_{M-1}(x) + \eta f_m(x)$$

Ordered Target Encoding in CatBoost

- Ordered target encoding refers to encoding a categorical value based on previous samples in the ordered sample set
- Steps:
  1. Randomly set an order for the samples
  2. For each category, maintain two running values
     1. TargetCount: Sum of target values for the same categorical feature value upto previous observation
     2. FeatureCount: Total number of observations with same categorical feature value upto previous observation
  3. Encoded value = (TargetCount+Prior)/(FeatureCount+1)
     Where prior is overall mean target

Order	Merchant Type	Target
1	Grocery	0
2	Grocery	0
3	Grocery	1
4	Electronics	1
5	Electronics	1
6	Electronics	1
7	LuxuryRare	1

Merchant Type	Encoded Valye	Target
Grocery	$\frac{0+0.7}{0+1}=0.6$	0
Grocery		0
Grocery		1
Electronics		1
Electronics		1
Electronics		1
LuxuryRare		1

How does CatBoost Solve Earlier Problems

- Rare categories
  - LuxuryRare has no previous history
  - CatBoost uses prior value instead of extreme encoding
- No label leakage
  - Each row does **not** use its own target
- More stable boosting
  - Early trees receive less exaggerated category signals

- Key Takeaway: CatBoost protects both the feature encoding and the boosting process itself

Summary of Tree Ensemble Methods

- A single decision tree is easy to interpret but can be unstable, high-variance, and prone to overfitting
- Ensembles improve performance by combining many trees. Two broad strategies are used:
  - Parallel ensembles: build trees independently to mainly reduce variance
  - Sequential ensembles: build trees stage by stage to correct earlier errors and reduce bias
- Bagging: bootstrap sampling + averaging/voting reduces prediction variance
- Random Forest: bagging + random feature selection at each split reduces correlation between trees and better generalization
- Boosting: builds an additive model of weak learners sequentially
  - AdaBoost: focuses more on misclassified points
  - Gradient Boosting: fits pseudo-residuals / negative gradients
  - XGBoost: regularized and computationally optimized gradient boosting
  - LightGBM: faster large-scale boosting using histogram splits and leaf-wise growth
  - CatBoost: handles categorical features more carefully using ordered statistics


operation == "MIRROR_X":
irror_mod.use_x = True
irror_mod.use_y = False
irror_mod.use_z = False
operation == "MIRROR_Y":
irror_mod.use_x = False
irror_mod.use_y = True
irror_mod.use_z = False
operation == "MIRROR_Z":
irror_mod.use_x = False
irror_mod.use_y = False
irror_mod.use_z = True
selection at the end -add
ob.select= 1
er_ob.select=1
ntext.scene.objects.acti
("Selected" + str(modifier
irror_ob.select = 0
bpy.context.selected_ob
ata.objects[one.name].se
int("please select exact

THANK YOU